Proposal for Orbital 26

Team Name:

[Please come up with a creative name for your team. Your team name should contain at least one English letter.]

Nala & Neb

Proposed Level of Achievement:

[Project Gemini or Apollo 11 or Artemis. Please read the descriptions of the levels of achievement on the Orbital website: https://soc-n.us/orbital-loa and choose only one.]

Artemis

Motivation

[Please describe the motivation behind your project.]

To allow students (particularly freshmen) to navigate within NUS, particularly on foot/bus to new venues.

Aim

[Please describe what you aim to achieve.]

The aim of this project is to develop a campus navigation and planning application tailored specifically for the National University of Singapore (NUS). While general navigation tools such as Google Maps provide basic routing within the campus, they often lack campus-specific data such as internal walkways, shuttle bus routes, and real-time classroom availability.

This system aims to provide an NUS-focused navigation solution that helps students efficiently move around campus and better plan their time between classes.

The application will support the following key capabilities:

Travel Time Estimation
Estimate approximate travel time between two user-selected locations within NUS.

Optimal Route Recommendation
Compute the shortest path between locations and recommend suitable transport modes such as walking, shuttle buses, or driving. The system may incorporate real-time shuttle bus information using the NUS NextBus API.

Location Bookmarking
Allow users to bookmark frequently visited locations (e.g., "Home", "Lecture Hall", "Library") for quick access.

Nearby Empty Classroom Finder
Help students locate nearby empty classrooms by integrating with the NUSMods API to determine classroom schedules.

(Optional) Reminder System
Provide reminders for upcoming classes based on saved locations and schedules.

(Optional) Home Screen Widgets
Provide quick-access widgets that allow users to view key information such as the time to their next class, nearby empty classrooms, or quick navigation to saved locations directly from their device home screen.

Overall, the goal is to build a campus-aware navigation assistant that improves mobility, convenience, and productivity for NUS students.

User Stories

[Please describe what the users would be able to do with your system.]

Navigation
As a student trying to reach my class, I want to find the fastest route between two locations on campus, so that I can arrive on time.

Transport Recommendation
As a student traveling across campus, I want to see recommended transport options (walking, bus, or driving) so that I can choose the most efficient route.

Bookmark Locations
As a frequent campus user, I want to save locations such as my home, tutorial rooms, or study spots, so that I can quickly navigate to them later.

Find Empty Classrooms
As a student looking for a study space, I want to find nearby classrooms that are currently empty, so that I can study or work there.

View Travel Time
As a student planning my schedule, I want to estimate travel time between my classes, so that I can plan my day more effectively.

(Optional) Class Reminder
As a student with multiple classes, I want to receive reminders before my next class, so that I do not forget or arrive late.

(Optional) Home Screen Widget
As a student who frequently checks campus information, I want to view important information such as travel time to my next class or nearby empty classrooms directly from my home screen, so that I can access key information quickly without opening the app.

Features

[Please list down the key features of your system with a rough timeline for completion.]

1. Feature 1 (Core): Campus Route Finder
   The system computes the shortest path between two locations in NUS using campus path data. It provides estimated travel time and route visualization.
2. Feature 2 (Core): Multi-Modal Transport Recommendation
   The system suggests appropriate travel modes such as walking, shuttle bus, or driving depending on the distance and route. Integration with the NUS NextBus API allows bus availability data to be considered.
3. Feature 3 (Core): Location Bookmarking
   Users can save frequently visited locations (e.g., "Home", "Classroom", "Study Spot") for quick navigation.
4. Feature 4 (Extension): Empty Classroom Finder
   Using the NUSMods API, the system identifies classrooms that are currently unoccupied and recommends nearby study locations.

5. Feature 5 (Extension): Smart Travel Time Estimation
   Travel time predictions are refined using bus schedules and estimated wait times.
6. Feature 6 (Extension): Class Reminder System
   Users can receive reminders for upcoming classes based on saved locations and schedule information.

Timeline

[Please provide a rough timeline for completion in terms of Milestones.]

7. Milestone 1 — Technical Proof of Concept
   (A minimal working system with basic frontend–backend integration)
   a. Basic map interface implemented
   b. Simple shortest path algorithm between two predefined campus locations
   c. Backend API integrated with frontend
   d. Demonstration of route computation working end-to-end
8. Milestone 2 — Prototype (A working system with core features)
   a. Fully functional route finder
   b. User input for start and destination locations
   c. Travel time estimation implemented
   d. Location bookmarking feature implemented
   e. Basic UI for route visualization
9. Milestone 3 — Extended System
   (A working system with both core and extension features)
   a. Integration with NUS NextBus API
   b. Empty classroom finder implemented using NUSMods API
   c. Improved travel time estimation
   d. Optional reminder & widget features implemented
   e. UI improvements and usability testing

Tech Stack

[Please list down the technologies that you are planning to use.]

1. Flutter (Dart) — Cross-platform framework used to develop the mobile application for both Android and iOS from a single codebase.
2. Flutter Widgets & Material/Cupertino UI — For building responsive user interfaces tailored to Android and iOS design conventions.
3. Node.js with Express.js — Backend server to handle API requests, routing computations, and integration with external APIs.
4. NUS NextBus API — Provides real-time shuttle bus information to improve route recommendations and travel time estimation.
5. NUSMods API — Used to retrieve classroom schedules and determine available classrooms.
6. Mapbox SDK / Google Maps SDK — For rendering the campus map, displaying routes, and handling user location data.
7. Device GPS / Location Services — To detect the user’s current location and recommend nearby locations or routes.
8. Graph Representation of Campus Pathways
9. Shortest Path Algorithms (e.g., Dijkstra or A)* to compute optimal routes within the campus.
10. Firebase / Cloud Firestore — Stores user preferences such as bookmarked locations and saved routes.
11. Firebase Cloud Messaging (FCM) — For class reminders and notifications.
12. Android App Widgets / iOS Widgets — For optional home screen widgets that display quick information such as travel time to the next class or nearby empty classrooms.

Qualifications

[Please elaborate on your technical skills and project experience of your team. This is required only for teams targeting Apollo 11 or Artemis.]

Benjamin:
CS1101S - A
CS1231S - A-
CS2040S - Pending

Alan:
CS1010 - A-
CS2040C - Pending
Linux server project - Jellyfin
Blog - blog.thealan.net

Software Engineering

[Please showcase the software engineering knowledge of your team by providing specific details about what software engineering related techniques / practices you plan to apply in the development process of your project. This is required only for teams targeting Apollo 11 or Artemis.]

The project will follow standard software engineering practices to ensure maintainability, scalability, and reliability.

Modular System Architecture
The system will separate frontend, backend, and data processing layers to allow independent development and easier debugging.

Version Control
Git will be used for version control, with feature branches and pull requests to manage collaborative development.

Agile Development Approach
Development will follow iterative milestones, allowing incremental implementation of features and continuous improvement.

Testing and Validation
Unit testing will be applied to critical components such as routing algorithms and API integration to ensure correctness.

Documentation
API endpoints, algorithms, and system architecture will be documented to ensure maintainability.